第 6 章　循环神经网络

经验是智慧之父，记忆是智慧之母.

——谚语

在前馈神经网络中，信息的传递是单向的，这种限制虽然使得网络变得更容易学习，但在一定程度上也减弱了神经网络模型的能力. 在生物神经网络中，神经元之间的连接关系要复杂得多. 前馈神经网络可以看作一个复杂的函数，每次输入都是独立的，即网络的输出只依赖于当前的输入. 但是在很多现实任务中，网络的输出不仅和当前时刻的输入相关，也和其过去一段时间的输出相关. 比如一个有限状态自动机，其下一个时刻的状态（输出）不仅仅和当前输入相关，也和当前状态（上一个时刻的输出）相关. 此外，前馈网络难以处理时序数据，比如视频、语音、文本等. 时序数据的长度一般是不固定的，而前馈神经网络要求输入和输出的维数都是固定的，不能任意改变. 因此，当处理这一类和时序数据相关的问题时，就需要一种能力更强的模型.

循环神经网络（Recurrent Neural Network，RNN）是一类具有短期记忆能力的神经网络. 在循环神经网络中，神经元不但可以接受其他神经元的信息，也可以接受自身的信息，形成具有环路的网络结构. 和前馈神经网络相比，循环神经网络更加符合生物神经网络的结构. 循环神经网络已经被广泛应用在语音识别、语言模型以及自然语言生成等任务上. 循环神经网络的参数学习可以通过随时间反向传播算法 [Werbos, 1990] 来学习. 随时间反向传播算法即按照时间的逆序将错误信息一步步地往前传递. 当输入序列比较长时，会存在梯度爆炸和消失问题 [Bengio et al., 1994; Hochreiter et al., 1997, 2001]，也称为长程依赖问题. 为了解决这个问题，人们对循环神经网络进行了很多的改进，其中最有效的改进方式引入门控机制（Gating Mechanism）.

此外，循环神经网络可以很容易地扩展到两种更广义的记忆网络模型：递归神经网络和图网络.

6.1 给网络增加记忆能力

为了处理这些时序数据并利用其历史信息，我们需要让网络具有短期记忆能力. 而前馈网络是一种静态网络，不具备这种记忆能力.

一般来讲，我们可以通过以下三种方法来给网络增加短期记忆能力.

此外，还有一种增加记忆能力的方法是引入外部记忆单元，参见第8.5节.

6.1.1 延时神经网络

一种简单的利用历史信息的方法是建立一个额外的延时单元，用来存储网络的历史信息（可以包括输入、输出、隐状态等）. 比较有代表性的模型是延时神经网络（Time Delay Neural Network，TDNN）[Lang et al., 1990; Waibel et al., 1989].

延时神经网络在时间维度上共享权值，以降低参数数量. 因此对于序列输入来讲，延时神经网络就相当于卷积神经网络.

延时神经网络是在前馈网络中的非输出层都添加一个延时器，记录神经元的最近几次活性值. 在第 t 个时刻，第 l 层神经元的活性值依赖于第 $l-1$ 层神经元的最近 K 个时刻的活性值，即

$$\boldsymbol{h}_t^{(l)} = f\left(\boldsymbol{h}_t^{(l-1)}, \boldsymbol{h}_{t-1}^{(l-1)}, \cdots, \boldsymbol{h}_{t-K}^{(l-1)}\right), \tag{6.1}$$

其中 $\boldsymbol{h}_t^{(l)} \in \mathbb{R}^{M_l}$ 表示第 l 层神经元在时刻 t 的活性值，M_l 为第 l 层神经元的数量. 通过延时器，前馈网络就具有了短期记忆的能力.

6.1.2 有外部输入的非线性自回归模型

自回归模型（AutoRegressive Model，AR）是统计学上常用的一类时间序列模型，用一个变量 $\boldsymbol{y}_t$ 的历史信息来预测自己.

$$\boldsymbol{y}_t = w_0 + \sum_{k=1}^{K} w_k \boldsymbol{y}_{t-k} + \epsilon_t, \tag{6.2}$$

其中 K 为超参数，$w_0, \cdots, w_K$ 为可学习参数，$\epsilon_t \sim \mathcal{N}(0, \sigma^2)$ 为第 t 个时刻的噪声，方差 σ^2 和时间无关.

有外部输入的非线性自回归模型（Nonlinear AutoRegressive with Exogenous Inputs Model，NARX）[Leontaritis et al., 1985] 是自回归模型的扩展，在每个时刻 t 都有一个外部输入 $\boldsymbol{x}_t$，产生一个输出 $\boldsymbol{y}_t$. NARX 通过一个延时器记录最近 K_x 次的外部输入和最近 K_y 次的输出，第 t 个时刻的输出 $\boldsymbol{y}_t$ 为

$$\boldsymbol{y}_t = f(\boldsymbol{x}_t, \boldsymbol{x}_{t-1}, \cdots, \boldsymbol{x}_{t-K_x}, \boldsymbol{y}_{t-1}, \boldsymbol{y}_{t-2}, \cdots, \boldsymbol{y}_{t-K_y}), \tag{6.3}$$

其中 $f(\cdot)$ 表示非线性函数，可以是一个前馈网络，K_x 和 K_y 为超参数.

6.1.3 循环神经网络

循环神经网络（Recurrent Neural Network，RNN）通过使用带自反馈的神经元，能够处理任意长度的时序数据.

给定一个输入序列 $\boldsymbol{x}_{1:T} = (\boldsymbol{x}_1, \boldsymbol{x}_2, \dots, \boldsymbol{x}_t, \dots, \boldsymbol{x}_T)$，循环神经网络通过下面公式更新带反馈边的隐藏层的活性值 $\boldsymbol{h}_t$：

$$\boldsymbol{h}_t = f(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t), \tag{6.4}$$

其中 $\boldsymbol{h}_0 = 0$，$f(\cdot)$ 为一个非线性函数，可以是一个前馈网络.

图6.1给出了循环神经网络的示例，其中“延时器”为一个虚拟单元，记录神经元的最近一次（或几次）活性值.

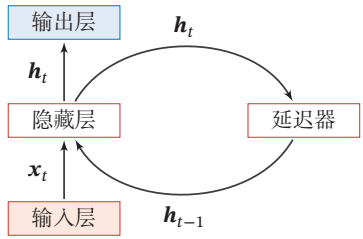


图 6.1 循环神经网络

从数学上讲，公式(6.4)可以看成一个动力系统. 因此，隐藏层的活性值 $\boldsymbol{h}_t$ 在很多文献上也称为状态（State）或隐状态（Hidden State）.

由于循环神经网络具有短期记忆能力，相当于存储装置，因此其计算能力十分强大. 理论上，循环神经网络可以近似任意的非线性动力系统（参见第6.2.1节）. 前馈神经网络可以模拟任何连续函数，而循环神经网络可以模拟任何程序.

6.2 简单循环网络

简单循环网络（Simple Recurrent Network，SRN）[Elman, 1990] 是一个非常简单的循环神经网络，只有一个隐藏层的神经网络. 在一个两层的前馈神经网络中，连接存在相邻的层与层之间，隐藏层的节点之间是无连接的. 而简单循环网络增加了从隐藏层到隐藏层的反馈连接.

令向量 $\boldsymbol{x}_t \in \mathbb{R}^M$ 表示在时刻 t 时网络的输入，$\boldsymbol{h}_t \in \mathbb{R}^D$ 表示隐藏层状态（即隐藏层神经元活性值），则 $\boldsymbol{h}_t$ 不仅和当前时刻的输入 $\boldsymbol{x}_t$ 相关，也和上一个时刻的隐藏层状态 $\boldsymbol{h}_{t-1}$ 相关. 简单循环网络在时刻 t 的更新公式为

$$\boldsymbol{z}_t = \boldsymbol{U}\boldsymbol{h}_{t-1} + \boldsymbol{W}\boldsymbol{x}_t + \boldsymbol{b}, \tag{6.5}$$

RNN 也经常被翻译为递归神经网络. 这里为了区别与另外一种递归神经网络（Recursive Neural Network，RecNN），我们称为循环神经网络.

动力系统（Dynamical System）是一个数学上的概念，指系统状态按照一定的规律随时间变化的系统. 具体地讲，动力系统是使用一个函数来描述一个给定空间（如某个物理系统的状态空间）中所有点随时间的变化情况. 生活中很多现象（比如钟摆晃动、台球轨迹等）都可以动力系统来描述.

$$\boldsymbol{h}_t = f(\boldsymbol{z}_t), \tag{6.6}$$

其中 $\boldsymbol{z}_t$ 为隐藏层的净输入，$\boldsymbol{U} \in \mathbb{R}^{D\times D}$ 为状态-状态权重矩阵，$\boldsymbol{W} \in \mathbb{R}^{D\times M}$ 为状态-输入权重矩阵，$\boldsymbol{b} \in \mathbb{R}^{D}$ 为偏置向量，$f(\cdot)$ 是非线性激活函数，通常为 Logistic 函数或 Tanh 函数. 公式 (6.5) 和公式 (6.6) 也经常直接写为

$$\boldsymbol{h}_t = f(\boldsymbol{U}\boldsymbol{h}_{t-1} + \boldsymbol{W}\boldsymbol{x}_t + \boldsymbol{b}). \tag{6.7}$$

如果我们把每个时刻的状态都看作前馈神经网络的一层，循环神经网络可以看作在时间维度上权值共享的神经网络. 图6.2给出了按时间展开的循环神经网络.

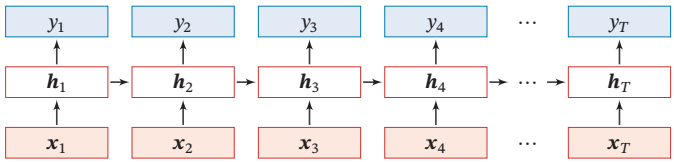


图 6.2 按时间展开的循环神经网络

6.2.1 循环神经网络的计算能力

我们先定义一个完全连接的循环神经网络，其输入为 $\boldsymbol{x}_t$，输出为 $\boldsymbol{y}_t$，

$$\boldsymbol{h}_t = f(\boldsymbol{U}\boldsymbol{h}_{t-1} + \boldsymbol{W}\boldsymbol{x}_t + \boldsymbol{b}), \tag{6.8}$$

$$\boldsymbol{y}_t = \boldsymbol{V}\boldsymbol{h}_t, \tag{6.9}$$

其中 $\boldsymbol{h}$ 为隐状态，$f(\cdot)$ 为非线性激活函数，$\boldsymbol{U}$、$\boldsymbol{W}$、$\boldsymbol{b}$ 和 $\boldsymbol{V}$ 为网络参数.

6.2.1.1 循环神经网络的通用近似定理

循环神经网络的拟合能力也十分强大. 一个完全连接的循环网络是任何非线性动力系统的近似器.

定理 6.1 – 循环神经网络的通用近似定理 [Haykin, 2009]： 如果一个完全连接的循环神经网络有足够数量的 sigmoid 型隐藏神经元，它可以以任意的准确率去近似任何一个非线性动力系统

$$\boldsymbol{s}_t = g(\boldsymbol{s}_{t-1}, \boldsymbol{x}_t), \tag{6.10}$$

$$\boldsymbol{y}_t = o(\boldsymbol{s}_t), \tag{6.11}$$

其中 $\boldsymbol{s}_t$ 为每个时刻的隐状态，$\boldsymbol{x}_t$ 是外部输入，$g(\cdot)$ 是可测的状态转换函数，$o(\cdot)$ 是连续输出函数，并且对状态空间的紧致性没有限制.

证明.（1）根据通用近似定理，两层的前馈神经网络可以近似任意有界闭集上的任意连续函数. 因此，动力系统的两个函数可以用两层的全连接前馈网络近似.

通用近似定理参见第4.3.1节.

首先，非线性动力系统的状态转换函数 $\boldsymbol{s}_t = g(\boldsymbol{s}_{t-1}, \boldsymbol{x}_t)$ 可以由一个两层的神经网络 $\boldsymbol{s}_t = \boldsymbol{C}f(\boldsymbol{A}\boldsymbol{s}_{t-1} + \boldsymbol{B}\boldsymbol{x}_t + \boldsymbol{b})$ 来近似，可以分解为

$$\boldsymbol{s}'_t = f(\boldsymbol{A}\boldsymbol{s}_{t-1} + \boldsymbol{B}\boldsymbol{x}_t + \boldsymbol{b}) \tag{6.12}$$

$$= f(\boldsymbol{A}\boldsymbol{C}\boldsymbol{s}'_{t-1} + \boldsymbol{B}\boldsymbol{x}_t + \boldsymbol{b}), \tag{6.13}$$

$$\boldsymbol{s}_t = \boldsymbol{C}\boldsymbol{s}'_t, \tag{6.14}$$

其中 $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}$ 为权重矩阵，$\boldsymbol{b}$ 为偏置向量.

本证明参考文献[Schäfer et al., 2006].

同理，非线性动力系统的输出函数 $\boldsymbol{y}_t = o(\boldsymbol{s}_t) = o(g(\boldsymbol{s}_{t-1}, \boldsymbol{x}_t))$ 也可以用一个两层的前馈神经网络近似.

$$\boldsymbol{y}'_t = f(\boldsymbol{A}'\boldsymbol{s}_{t-1} + \boldsymbol{B}'\boldsymbol{x}_t + \boldsymbol{b}') \tag{6.15}$$

$$= f(\boldsymbol{A}'\boldsymbol{C}\boldsymbol{s}'_{t-1} + \boldsymbol{B}'\boldsymbol{x}_t + \boldsymbol{b}'), \tag{6.16}$$

$$\boldsymbol{y}_t = \boldsymbol{D}\boldsymbol{y}'_t, \tag{6.17}$$

其中 A', B', D 为权重矩阵，$\boldsymbol{b}'$ 为偏置向量.

（2）公式(6.13)和公式(6.16)可以合并为

$$\begin{bmatrix} \boldsymbol{s}'_t \\ \boldsymbol{y}'_t \end{bmatrix} = f\left(\begin{bmatrix} \boldsymbol{A}\boldsymbol{C} & 0 \\ \boldsymbol{A}'\boldsymbol{C} & 0 \end{bmatrix}\begin{bmatrix} \boldsymbol{s}'_{t-1} \\ \boldsymbol{y}'_{t-1} \end{bmatrix} + \begin{bmatrix} \boldsymbol{B} \\ \boldsymbol{B}' \end{bmatrix}\boldsymbol{x}_t + \begin{bmatrix} \boldsymbol{b} \\ \boldsymbol{b}' \end{bmatrix}\right). \tag{6.18}$$

公式(6.17)可以改写为

$$\boldsymbol{y}_t = \begin{bmatrix} 0 & \boldsymbol{D} \end{bmatrix}\begin{bmatrix} \boldsymbol{s}'_t \\ \boldsymbol{y}'_t \end{bmatrix}. \tag{6.19}$$

令 $\boldsymbol{h}_t = [\boldsymbol{s}'_t; \boldsymbol{y}'_t]$，则非线性动力系统可以由下面的全连接循环神经网络来近似.

$$\boldsymbol{h}_t = f(\boldsymbol{U}\boldsymbol{h}_{t-1} + \boldsymbol{W}\boldsymbol{x}_t + \boldsymbol{b}), \tag{6.20}$$

$$\boldsymbol{y}_t = \boldsymbol{V}\boldsymbol{h}_t, \tag{6.21}$$

其中 $\boldsymbol{U} = \begin{bmatrix} \boldsymbol{A}\boldsymbol{C} & 0 \\ \boldsymbol{A}'\boldsymbol{C} & 0 \end{bmatrix}, \boldsymbol{W} = \begin{bmatrix} \boldsymbol{B} \\ \boldsymbol{B}' \end{bmatrix}, \boldsymbol{b} = \begin{bmatrix} \boldsymbol{b} \\ \boldsymbol{b}' \end{bmatrix}, \boldsymbol{V} = \begin{bmatrix} 0 & \boldsymbol{D} \end{bmatrix}$. □

6.2.1.2 图灵完备

图灵完备（Turing Completeness）是指一种数据操作规则，比如一种计算机编程语言，可以实现图灵机（Turing Machine）的所有功能，解决所有的可计算问题. 目前主流的编程语言（比如 C++、Java、Python 等）都是图灵完备的.

图灵机是一种抽象的信息处理装置，可以用来解决所有的可计算问题，参见第8.5.2节.

定理 6.2 – 图灵完备 [Siegelmann et al., 1991]： 所有的图灵机都可以被一个由使用 Sigmoid 型激活函数的神经元构成的全连接循环网络来进行模拟.

因此，一个完全连接的循环神经网络可以近似解决所有的可计算问题.

6.3 应用到机器学习

循环神经网络可以应用到很多不同类型的机器学习任务. 根据这些任务的特点可以分为以下几种模式：序列到类别模式、同步的序列到序列模式、异步的序列到序列模式.

下面我们分别来看下这几种应用模式.

6.3.1 序列到类别模式

序列到类别模式主要用于序列数据的分类问题：输入为序列，输出为类别. 比如在文本分类中，输入数据为单词的序列，输出为该文本的类别.

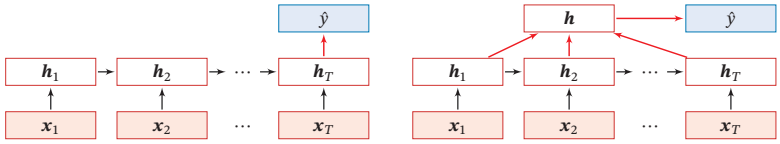
假设一个样本 $\boldsymbol{x}_{1:T} = (\boldsymbol{x}_1, \cdots, \boldsymbol{x}_T)$ 为一个长度为 T 的序列，输出为一个类别 $y \in \{1, \cdots, C\}$. 我们可以将样本 $\boldsymbol{x}$ 按不同时刻输入到循环神经网络中，并得到不同时刻的隐藏状态 $\boldsymbol{h}_1, \cdots, \boldsymbol{h}_T$. 我们可以将 $\boldsymbol{h}_T$ 看作整个序列的最终表示（或特征），并输入给分类器 $g(\cdot)$ 进行分类（如图6.3a所示），即

$$\hat{y} = g(\boldsymbol{h}_T), \tag{6.22}$$

其中 $g(\cdot)$ 可以是简单的线性分类器（比如 Logistic 回归）或复杂的分类器（比如多层前馈神经网络）.

除了将最后时刻的状态作为整个序列的表示之外，我们还可以对整个序列的所有状态进行平均，并用这个平均状态来作为整个序列的表示（如图6.3b所示），即

$$\hat{y} = g\Big(\frac{1}{T}\sum_{t=1}^{T}\boldsymbol{h}_t\Big). \tag{6.23}$$



(a) 正常模式 (b) 按时间进行平均采样模式

图 6.3 序列到类别模式

6.3.2 同步的序列到序列模式

同步的序列到序列模式主要用于序列标注（Sequence Labeling）任务，即每一时刻都有输入和输出，输入序列和输出序列的长度相同. 比如在词性标注（Part-of-Speech Tagging）中，每一个单词都需要标注其对应的词性标签.

在同步的序列到序列模式（如图6.4所示）中，输入为一个长度为 T 的序列 $\boldsymbol{x}_{1:T} = (\boldsymbol{x}_1, \cdots, \boldsymbol{x}_T)$，输出为序列 $y_{1:T} = (y_1, \cdots, y_T)$. 样本 $\boldsymbol{x}$ 按不同时刻输入到循环神经网络中，并得到不同时刻的隐状态 $\boldsymbol{h}_1, \cdots, \boldsymbol{h}_T$. 每个时刻的隐状态 $\boldsymbol{h}_t$ 代表了当前时刻和历史的信息，并输入给分类器 $g(\cdot)$ 得到当前时刻的标签 $\hat{y}_t$，即

$$\hat{y}_t = g(\boldsymbol{h}_t), \qquad \forall t \in [1, T]. \tag{6.24}$$

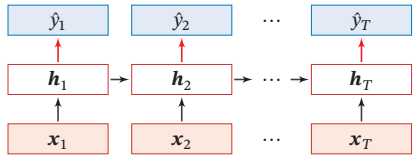


图 6.4 同步的序列到序列模式

6.3.3 异步的序列到序列模式

异步的序列到序列模式也称为编码器-解码器（Encoder-Decoder）模型，即输入序列和输出序列不需要有严格的对应关系，也不需要保持相同的长度. 比如在机器翻译中，输入为源语言的单词序列，输出为目标语言的单词序列.

参见第15.6节.

在异步的序列到序列模式中，输入为长度为 T 的序列 $\boldsymbol{x}_{1:T} = (\boldsymbol{x}_1, \cdots, \boldsymbol{x}_T)$，输出为长度为 M 的序列 $y_{1:M} = (y_1, \cdots, y_M)$. 异步的序列到序列模式一般通过先编码后解码的方式来实现. 先将样本 $\boldsymbol{x}$ 按不同时刻输入到一个循环神经网络（编码器）中，并得到其编码 $\boldsymbol{h}_T$. 然后再使用另一个循环神经网络（解码器），得到输出序列 $\hat{y}_{1:M}$. 为了建立输出序列之间的依赖关系，在解码器中通常使用非线性的自回归模型. 令 $f_1(\cdot)$ 和 $f_2(\cdot)$ 分别为用作编码器和解码器的循环神经网络，则编码

器-解码器模型可以写为

$$\boldsymbol{h}_t = f_1(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t), \qquad \forall t \in [1, T] \tag{6.25}$$

$$\boldsymbol{h}_{T+t} = f_2(\boldsymbol{h}_{T+t-1}, \hat{\boldsymbol{y}}_{t-1}), \qquad \forall t \in [1, M] \tag{6.26}$$

$$\hat{\boldsymbol{y}}_t = g(\boldsymbol{h}_{T+t}), \qquad \forall t \in [1, M] \tag{6.27}$$

其中 $g(\cdot)$ 为分类器，$\hat{\boldsymbol{y}}_t$ 为预测输出 $\hat{y}_t$ 的向量表示. 在解码器通常采用自回归模型，每个时刻的输入为上一时刻的预测结果 $\hat{y}_{t-1}$.

自回归模型参见第6.1.2节.

图6.5给出了异步的序列到序列模式示例，其中 $\langle EOS \rangle$ 表示输入序列的结束，虚线表示将上一个时刻的输出作为下一个时刻的输入.

异步的序列到序列模式可以进一步参见第15.6.1节.

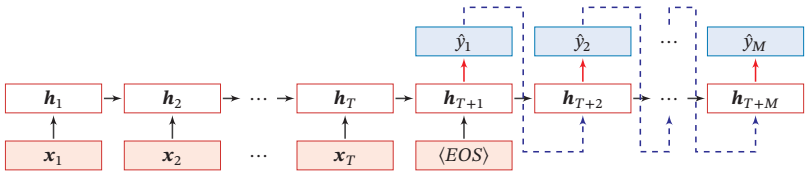


图 6.5 异步的序列到序列模式

6.4 参数学习

循环神经网络的参数可以通过梯度下降方法来进行学习.

以随机梯度下降为例，给定一个训练样本 $(\boldsymbol{x}, \boldsymbol{y})$，其中 $\boldsymbol{x}_{1:T} = (\boldsymbol{x}_1, \cdots, \boldsymbol{x}_T)$ 为长度是 T 的输入序列，$y_{1:T} = (y_1, \cdots, y_T)$ 是长度为 T 的标签序列. 即在每个时刻 t，都有一个监督信息 y_t，我们定义时刻 t 的损失函数为

不失一般性，这里我们以同步的序列到序列模式为例来介绍循环神经网络的参数学习.

$$\mathcal{L}_t = \mathcal{L}(y_t, g(\boldsymbol{h}_t)), \tag{6.28}$$

其中 $g(\boldsymbol{h}_t)$ 为第 t 时刻的输出，$\mathcal{L}$ 为可微分的损失函数，比如交叉熵. 那么整个序列的损失函数为

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t. \tag{6.29}$$

整个序列的损失函数 $\mathcal{L}$ 关于参数 $\boldsymbol{U}$ 的梯度为

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{U}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{U}}, \tag{6.30}$$

即每个时刻损失 $\mathcal{L}_t$ 对参数 $\boldsymbol{U}$ 的偏导数之和.

循环神经网络中存在一个递归调用的函数 $f(\cdot)$，因此其计算参数梯度的方式和前馈神经网络不太相同. 在循环神经网络中主要有两种计算梯度的方式：随时间反向传播（BPTT）算法和实时循环学习（RTRL）算法.

6.4.1 随时间反向传播算法

随时间反向传播（BackPropagation Through Time，BPTT）算法的主要思想是通过类似前馈神经网络的误差反向传播算法[Werbos, 1990]来计算梯度.

BPTT 算法将循环神经网络看作一个展开的多层前馈网络，其中“每一层”对应循环网络中的“每个时刻”（见图6.2）. 这样，循环神经网络就可以按照前馈网络中的反向传播算法计算参数梯度. 在“展开”的前馈网络中，所有层的参数是共享的，因此参数的真实梯度是所有“展开层”的参数梯度之和.

计算偏导数 $\frac{\partial \mathcal{L}_t}{\partial \boldsymbol{U}}$ 先来计算公式(6.30)中第 t 时刻损失对参数 $\boldsymbol{U}$ 的偏导数 $\frac{\partial \mathcal{L}_t}{\partial \boldsymbol{U}}$.

因为参数 $\boldsymbol{U}$ 和隐藏层在每个时刻 $k(1 \leq k \leq t)$ 的净输入 $\boldsymbol{z}_k = \boldsymbol{U}\boldsymbol{h}_{k-1} + \boldsymbol{W}\boldsymbol{x}_k + \boldsymbol{b}$ 有关，因此第 t 时刻的损失函数 $\mathcal{L}_t$ 关于参数 u_{ij} 的梯度为：

$$\frac{\partial \mathcal{L}_t}{\partial u_{ij}} = \sum_{k=1}^{t} \frac{\partial^+ \boldsymbol{z}_k}{\partial u_{ij}} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{z}_k}, \tag{6.31}$$

链式法则参见公式(B.18).

其中 $\frac{\partial^+ \boldsymbol{z}_k}{\partial u_{ij}}$ 表示“直接”偏导数，即公式 $\boldsymbol{z}_k = \boldsymbol{U}\boldsymbol{h}_{k-1} + \boldsymbol{W}\boldsymbol{x}_k + \boldsymbol{b}$ 中保持 $\boldsymbol{h}_{k-1}$ 不变，对 u_{ij} 进行求偏导数，得到

$$\frac{\partial^+ \boldsymbol{z}_k}{\partial u_{ij}} = [0, \cdots, [\boldsymbol{h}_{k-1}]_j, \cdots, 0] \tag{6.32}$$

$$\triangleq \mathbb{I}_i([\boldsymbol{h}_{k-1}]_j), \tag{6.33}$$

其中 $[\boldsymbol{h}_{k-1}]_j$ 为第 $k-1$ 时刻隐状态的第 j 维；$\mathbb{I}_i(x)$ 除了第 i 个元素的值为 x 外，其余都为 0 的行向量.

定义误差项 $\delta_{t,k} = \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{z}_k}$ 为第 t 时刻的损失对第 k 时刻隐藏神经层的净输入 $\boldsymbol{z}_k$ 的导数，则当 $1 \leq k < t$ 时

$$\delta_{t,k} = \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{z}_k} \tag{6.34}$$

$$= \frac{\partial \boldsymbol{h}_k}{\partial \boldsymbol{z}_k} \frac{\partial \boldsymbol{z}_{k+1}}{\partial \boldsymbol{h}_k} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{z}_{k+1}} \tag{6.35}$$

$$= \mathrm{diag}(f'(\boldsymbol{z}_k))\boldsymbol{U}^{\mathrm{T}}\delta_{t,k+1}. \tag{6.36}$$

将公式(6.36)和公式(6.33)代入公式(6.31)得到

$$\frac{\partial \mathcal{L}_t}{\partial u_{ij}} = \sum_{k=1}^{t} [\delta_{t,k}]_i [\boldsymbol{h}_{k-1}]_j. \tag{6.37}$$

将上式写成矩阵形式为

$$\frac{\partial \mathcal{L}_t}{\partial \boldsymbol{U}} = \sum_{k=1}^{t} \delta_{t,k} \boldsymbol{h}_{k-1}^{\mathrm{T}}. \tag{6.38}$$

图6.6给出了误差项随时间进行反向传播算法的示例.

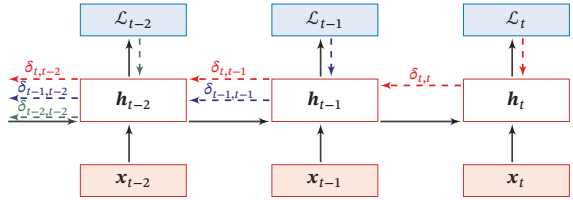


图 6.6 误差项随时间反向传播算法示例

参数梯度 将公式(6.38)代入到公式(6.30),得到整个序列的损失函数 $\mathcal{L}$ 关于参数 $\boldsymbol{U}$ 的梯度

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{U}} = \sum_{t=1}^{T}\sum_{k=1}^{t} \delta_{t,k}\boldsymbol{h}_{k-1}^{\mathrm{T}}. \tag{6.39}$$

同理可得,$\mathcal{L}$ 关于权重 $\boldsymbol{W}$ 和偏置 $\boldsymbol{b}$ 的梯度为

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}} = \sum_{t=1}^{T}\sum_{k=1}^{t} \delta_{t,k}\boldsymbol{x}_{k}^{\mathrm{T}}, \tag{6.40}$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}} = \sum_{t=1}^{T}\sum_{k=1}^{t} \delta_{t,k}. \tag{6.41}$$

计算复杂度 在BPTT算法中,参数的梯度需要在一个完整的“前向”计算和“反向”计算后才能得到并进行参数更新.

6.4.2 实时循环学习算法

与反向传播的BPTT算法不同的是,实时循环学习(Real-Time Recurrent Learning,RTRL)是通过前向传播的方式来计算梯度 [Williams et al., 1995].

梯度前向传播可以参考自动微分中的前向模式,参见第4.5.3节.

假设循环神经网络中第 $t+1$ 时刻的状态 $\boldsymbol{h}_{t+1}$ 为

$$\boldsymbol{h}_{t+1} = f(\boldsymbol{z}_{t+1}) = f(\boldsymbol{U}\boldsymbol{h}_t + \boldsymbol{W}\boldsymbol{x}_{t+1} + \boldsymbol{b}), \tag{6.42}$$

其关于参数 u_{ij} 的偏导数为

$$\frac{\partial \boldsymbol{h}_{t+1}}{\partial u_{ij}} = \left(\frac{\partial^{+}\boldsymbol{z}_{t+1}}{\partial u_{ij}} + \frac{\partial \boldsymbol{h}_t}{\partial u_{ij}}\boldsymbol{U}^{\mathrm{T}}\right)\frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{z}_{t+1}} \tag{6.43}$$

$$= \left(\mathbb{I}_i([\boldsymbol{h}_t]_j) + \frac{\partial \boldsymbol{h}_t}{\partial u_{ij}}\boldsymbol{U}^{\mathrm{T}}\right)\mathrm{diag}(f'(\boldsymbol{z}_{t+1})) \tag{6.44}$$

$$= \left(\mathbb{I}_i([\boldsymbol{h}_t]_j) + \frac{\partial \boldsymbol{h}_t}{\partial u_{ij}} \boldsymbol{U}^{\mathrm{T}}\right) \odot \left(f'(\boldsymbol{z}_{t+1})\right)^{\mathrm{T}}, \tag{6.45}$$

其中 $\mathbb{I}_i(x)$ 是除了第 i 行值为 x 外，其余都为 0 的行向量.

RTRL 算法从第 1 个时刻开始，除了计算循环神经网络的隐状态之外，还利用公式 (6.45) 依次前向计算偏导数 $\frac{\partial \boldsymbol{h}_1}{\partial u_{ij}}, \frac{\partial \boldsymbol{h}_2}{\partial u_{ij}}, \frac{\partial \boldsymbol{h}_3}{\partial u_{ij}}, \cdots$.

这样，假设第 t 个时刻存在一个监督信息，其损失函数为 $\mathcal{L}_t$，就可以同时计算损失函数对 u_{ij} 的偏导数

$$\frac{\partial \mathcal{L}_t}{\partial u_{ij}} = \frac{\partial \boldsymbol{h}_t}{\partial u_{ij}} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{h}_t}. \tag{6.46}$$

这样在第 t 时刻，可以实时地计算损失 $\mathcal{L}_t$ 关于参数 $\boldsymbol{U}$ 的梯度，并更新参数. 参数 $\boldsymbol{W}$ 和 $\boldsymbol{b}$ 的梯度也可以同样按上述方法实时计算.

两种算法比较 RTRL 算法和 BPTT 算法都是基于梯度下降的算法，分别通过前向模式和反向模式应用链式法则来计算梯度. 在循环神经网络中，一般网络输出维度远低于输入维度，因此 BPTT 算法的计算量会更小，但是 BPTT 算法需要保存所有时刻的中间梯度，空间复杂度较高. RTRL 算法不需要梯度回传，因此非常适合用于需要在线学习或无限序列的任务中.

6.5 长程依赖问题

循环神经网络在学习过程中的主要问题是由于梯度消失或爆炸问题，很难建模长时间间隔（Long Range）的状态之间的依赖关系.

在 BPTT 算法中，将公式 (6.36) 展开得到

$$\delta_{t,k} = \prod_{\tau=k}^{t-1} \left(\mathrm{diag}(f'(\boldsymbol{z}_\tau))\boldsymbol{U}^{\mathrm{T}}\right)\delta_{t,t}. \tag{6.47}$$

如果定义 $\gamma \cong \|\mathrm{diag}(f'(\boldsymbol{z}_\tau))\boldsymbol{U}^{\mathrm{T}}\|$，则

$$\delta_{t,k} \cong \gamma^{t-k}\delta_{t,t}. \tag{6.48}$$

若 $\gamma > 1$，当 $t-k \to \infty$ 时，$\gamma^{t-k} \to \infty$. 当间隔 $t-k$ 比较大时，梯度也变得很大，会造成系统不稳定，称为梯度爆炸问题（Gradient Exploding Problem）.

相反，若 $\gamma < 1$，当 $t-k \to \infty$ 时，$\gamma^{t-k} \to 0$. 当间隔 $t-k$ 比较大时，梯度也变得非常小，会出现和深层前馈神经网络类似的梯度消失问题（Vanishing Gradient Problem）.

要注意的是，在循环神经网络中的梯度消失不是说 $\frac{\partial \mathcal{L}_t}{\partial \boldsymbol{U}}$ 的梯度消失了，而是 $\frac{\partial \mathcal{L}_t}{\partial \boldsymbol{h}_k}$ 的梯度消失了（当间隔 $t-k$ 比较大时）. 也就是说，参数 $\boldsymbol{U}$ 的更新主要靠当前时刻 t 的几个相邻状态 $\boldsymbol{h}_k$ 来更新，长距离的状态对参数 $\boldsymbol{U}$ 没有影响.

由于循环神经网络经常使用非线性激活函数为Logistic函数或Tanh函数作为非线性激活函数，其导数值都小于1，并且权重矩阵 $\|\boldsymbol{U}\|$ 也不会太大，因此如果时间间隔 $t-k$ 过大，$\delta_{t,k}$ 会趋向于0，因而经常会出现梯度消失问题.

虽然简单循环网络理论上可以建立长时间间隔的状态之间的依赖关系，但是由于梯度爆炸或消失问题，实际上只能学习到短期的依赖关系. 这样，如果时刻 t 的输出 y_t 依赖于时刻 k 的输入 $\boldsymbol{x}_k$，当间隔 $t-k$ 比较大时，简单神经网络很难建模这种长距离的依赖关系，称为长程依赖问题（Long-Term Dependencies Problem）.

长程依赖问题也称为长期依赖问题或长距离依赖问题.

6.5.1 改进方案

为了避免梯度爆炸或消失问题，一种最直接的方式就是选取合适的参数，同时使用非饱和的激活函数，尽量使得 $\text{diag}(f'(\boldsymbol{z}))\boldsymbol{U}^{\mathrm{T}} \approx 1$，这种方式需要足够的人工调参经验，限制了模型的广泛应用. 比较有效的方式是通过改进模型或优化方法来缓解循环网络的梯度爆炸和梯度消失问题.

梯度爆炸 一般而言，循环网络的梯度爆炸问题比较容易解决，一般通过权重衰减或梯度截断来避免.

梯度截断是一种启发式的解决梯度爆炸问题的有效方法，参见第7.2.4.4节.

权重衰减是通过给参数增加 ℓ_1 或 ℓ_2 范数的正则化项来限制参数的取值范围，从而使得 $\gamma \leq 1$. 梯度截断是另一种有效的启发式方法，当梯度的模大于一定阈值时，就将它截断成为一个较小的数.

梯度消失 梯度消失是循环网络的主要问题. 除了使用一些优化技巧外，更有效的方式就是改变模型，比如让 $\boldsymbol{U}=\boldsymbol{I}$，同时令 $\frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{h}_{t-1}} = \boldsymbol{I}$ 为单位矩阵，即

$$\boldsymbol{h}_t = \boldsymbol{h}_{t-1} + g(\boldsymbol{x}_t; \theta), \tag{6.49}$$

其中 $g(\cdot)$ 是一个非线性函数，θ 为参数.

公式(6.49)中，$\boldsymbol{h}_t$ 和 $\boldsymbol{h}_{t-1}$ 之间为线性依赖关系，且权重系数为1，这样就不存在梯度爆炸或消失问题. 但是，这种改变也丢失了神经元在反馈边上的非线性激活的性质，因此也降低了模型的表示能力.

为了避免这个缺点，我们可以采用一种更加有效的改进策略：

$$\boldsymbol{h}_t = \boldsymbol{h}_{t-1} + g(\boldsymbol{x}_t, \boldsymbol{h}_{t-1}; \theta), \tag{6.50}$$

这种改进策略和残差连接的思想十分类似，参见第5.4.4节.

这样 $\boldsymbol{h}_t$ 和 $\boldsymbol{h}_{t-1}$ 之间为既有线性关系，也有非线性关系，并且可以缓解梯度消失问题. 但这种改进依然存在两个问题：

（1）梯度爆炸问题：令 $\boldsymbol{z}_k = \boldsymbol{U}\boldsymbol{h}_{k-1} + \boldsymbol{W}\boldsymbol{x}_k + \boldsymbol{b}$ 为在第 k 时刻函数 $g(\cdot)$ 的输入，在计算公式(6.34)中的误差项 $\delta_{t,k} = \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{z}_k}$ 时，梯度可能会过大，从而导致梯度爆炸问题.

参见习题6-3.

（2）记忆容量（Memory Capacity）问题：随着 $\boldsymbol{h}_t$ 不断累积存储新的输入信息，会发生饱和现象. 假设 $g(\cdot)$ 为 Logistic 函数，则随着时间 t 的增长，$\boldsymbol{h}_t$ 会变得越来越大，从而导致 $\boldsymbol{h}$ 变得饱和. 也就是说，隐状态 $\boldsymbol{h}_t$ 可以存储的信息是有限的，随着记忆单元存储的内容越来越多，其丢失的信息也越来越多.

还有一种增加记忆容量的方法是增加一些额外的存储单元：外部记忆，参见第8.5节.

为了解决这两个问题，可以通过引入门控机制来进一步改进模型.

6.6 基于门控的循环神经网络

为了改善循环神经网络的长程依赖问题，一种非常好的解决方案是在公式(6.50)的基础上引入门控机制来控制信息的累积速度，包括有选择地加入新的信息，并有选择地遗忘之前累积的信息. 这一类网络可以称为基于门控的循环神经网络（Gated RNN）. 本节中，主要介绍两种基于门控的循环神经网络：长短期记忆网络和门控循环单元网络.

6.6.1 长短期记忆网络

长短期记忆网络（Long Short-Term Memory Network，LSTM）[Gers et al., 2000; Hochreiter et al., 1997] 是循环神经网络的一个变体，可以有效地解决简单循环神经网络的梯度爆炸或消失问题.

在公式(6.50)的基础上，LSTM 网络主要改进在以下两个方面：

新的内部状态 LSTM 网络引入一个新的内部状态（internal state）$\boldsymbol{c}_t \in \mathbb{R}^D$ 专门进行线性的循环信息传递，同时（非线性地）输出信息给隐藏层的外部状态 $\boldsymbol{h}_t \in \mathbb{R}^D$. 内部状态 $\boldsymbol{c}_t$ 通过下面公式计算：

$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t, \tag{6.51}$$

$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t), \tag{6.52}$$

其中 $\boldsymbol{f}_t \in [0,1]^D$、$\boldsymbol{i}_t \in [0,1]^D$ 和 $\boldsymbol{o}_t \in [0,1]^D$ 为三个门（gate）来控制信息传递的路径；$\odot$ 为向量元素乘积；$\boldsymbol{c}_{t-1}$ 为上一时刻的记忆单元；$\tilde{\boldsymbol{c}}_t \in \mathbb{R}^D$ 是通过非线性函数得到的候选状态：

$$\tilde{\boldsymbol{c}}_t = \tanh(\boldsymbol{W}_c\boldsymbol{x}_t + \boldsymbol{U}_c\boldsymbol{h}_{t-1} + \boldsymbol{b}_c). \tag{6.53}$$

公式(6.53)~公式(6.56)中的 $\boldsymbol{W}_*, \boldsymbol{U}_*, \boldsymbol{b}_*$ 为可学习的网络参数，其中 $* \in \{i, f, o, c\}$.

在每个时刻 t，LSTM 网络的内部状态 $\boldsymbol{c}_t$ 记录了到当前时刻为止的历史信息.

门控机制 在数字电路中，门（gate）为一个二值变量 $\{0,1\}$，0 代表关闭状态，不许任何信息通过；1 代表开放状态，允许所有信息通过.

LSTM 网络引入门控机制（Gating Mechanism）来控制信息传递的路径. 公式 (6.51) 和公式 (6.52) 中三个“门”分别为输入门 $\boldsymbol{i}_t$、遗忘门 $\boldsymbol{f}_t$ 和输出门 $\boldsymbol{o}_t$. 这三个门的作用为

（1） 遗忘门 $\boldsymbol{f}_t$ 控制上一个时刻的内部状态 $\boldsymbol{c}_{t-1}$ 需要遗忘多少信息.

（2） 输入门 $\boldsymbol{i}_t$ 控制当前时刻的候选状态 $\tilde{\boldsymbol{c}}_t$ 有多少信息需要保存.

（3） 输出门 $\boldsymbol{o}_t$ 控制当前时刻的内部状态 $\boldsymbol{c}_t$ 有多少信息需要输出给外部状态 $\boldsymbol{h}_t$.

当 $\boldsymbol{f}_t = \boldsymbol{0}, \boldsymbol{i}_t = \boldsymbol{1}$ 时，记忆单元将历史信息清空，并将候选状态向量 $\tilde{\boldsymbol{c}}_t$ 写入. 但此时记忆单元 $\boldsymbol{c}_t$ 依然和上一时刻的历史信息相关. 当 $\boldsymbol{f}_t = \boldsymbol{1}, \boldsymbol{i}_t = \boldsymbol{0}$ 时，记忆单元将复制上一时刻的内容，不写入新的信息.

LSTM 网络中的“门”是一种“软”门，取值在 $(0,1)$ 之间，表示以一定的比例允许信息通过. 三个门的计算方式为：

$$\boldsymbol{i}_t = \sigma(\boldsymbol{W}_i \boldsymbol{x}_t + \boldsymbol{U}_i \boldsymbol{h}_{t-1} + \boldsymbol{b}_i), \tag{6.54}$$

$$\boldsymbol{f}_t = \sigma(\boldsymbol{W}_f \boldsymbol{x}_t + \boldsymbol{U}_f \boldsymbol{h}_{t-1} + \boldsymbol{b}_f), \tag{6.55}$$

$$\boldsymbol{o}_t = \sigma(\boldsymbol{W}_o \boldsymbol{x}_t + \boldsymbol{U}_o \boldsymbol{h}_{t-1} + \boldsymbol{b}_o), \tag{6.56}$$

其中 $\sigma(\cdot)$ 为 Logistic 函数，其输出区间为 $(0,1)$，$\boldsymbol{x}_t$ 为当前时刻的输入，$\boldsymbol{h}_{t-1}$ 为上一时刻的外部状态.

图6.7给出了 LSTM 网络的循环单元结构，其计算过程为：1）首先利用上一时刻的外部状态 $\boldsymbol{h}_{t-1}$ 和当前时刻的输入 $\boldsymbol{x}_t$，计算出三个门，以及候选状态 $\tilde{\boldsymbol{c}}_t$；2）结合遗忘门 $\boldsymbol{f}_t$ 和输入门 $\boldsymbol{i}_t$ 来更新记忆单元 $\boldsymbol{c}_t$；3）结合输出门 $\boldsymbol{o}_t$，将内部状态的信息传递给外部状态 $\boldsymbol{h}_t$.

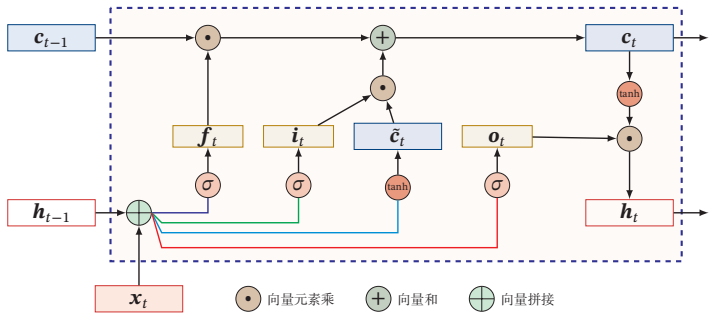


图 6.7 LSTM 网络的循环单元结构

通过 LSTM 循环单元，整个网络可以建立较长距离的时序依赖关系. 公式 (6.51)~公式 (6.56) 可以简洁地描述为

$$\begin{bmatrix} \tilde{\boldsymbol{c}}_t \\ \boldsymbol{o}_t \\ \boldsymbol{i}_t \\ \boldsymbol{f}_t \end{bmatrix} = \begin{bmatrix} \tanh \\ \sigma \\ \sigma \\ \sigma \end{bmatrix} \left(\boldsymbol{W} \begin{bmatrix} \boldsymbol{x}_t \\ \boldsymbol{h}_{t-1} \end{bmatrix} + \boldsymbol{b} \right), \tag{6.57}$$

$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t, \tag{6.58}$$

$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t), \tag{6.59}$$

其中 $\boldsymbol{x}_t \in \mathbb{R}^M$ 为当前时刻的输入，$\boldsymbol{W} \in \mathbb{R}^{4D \times (M+D)}$ 和 $\boldsymbol{b} \in \mathbb{R}^{4D}$ 为网络参数.

记忆 循环神经网络中的隐状态 $\boldsymbol{h}$ 存储了历史信息，可以看作一种记忆（Memory）. 在简单循环网络中，隐状态每个时刻都会被重写，因此可以看作一种短期记忆（Short-Term Memory）. 在神经网络中，长期记忆（Long-Term Memory）可以看作网络参数，隐含了从训练数据中学到的经验，其更新周期要远远慢于短期记忆. 而在 LSTM 网络中，记忆单元 $\boldsymbol{c}$ 可以在某个时刻捕捉到某个关键信息，并有能力将此关键信息保存一定的时间间隔. 记忆单元 $\boldsymbol{c}$ 中保存信息的生命周期要长于短期记忆 $\boldsymbol{h}$，但又远远短于长期记忆，因此称为长短期记忆（Long Short-Term Memory）.

长短期记忆是指长的“短期记忆”.



一般在深度网络参数学习时，参数初始化的值一般都比较小. 但是在训练 LSTM 网络时，过小的值会使得遗忘门的值比较小. 这意味着前一时刻的信息大部分都丢失了，这样网络很难捕捉到长距离的依赖信息. 并且相邻时间间隔的梯度会非常小，这会导致梯度弥散问题. 因此遗忘的参数初始值一般都设得比较大，其偏置向量 $\boldsymbol{b}_f$ 设为 1 或 2.

6.6.2 LSTM 网络的各种变体

目前主流的 LSTM 网络用三个门来动态地控制内部状态应该遗忘多少历史信息，输入多少新信息，以及输出多少信息. 我们可以对门控机制进行改进并获得 LSTM 网络的不同变体.

无遗忘门的 LSTM 网络 [Hochreiter et al., 1997] 最早提出的 LSTM 网络是没有遗忘门的，其内部状态的更新为

$$\boldsymbol{c}_t = \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t. \tag{6.60}$$

如之前的分析，记忆单元 $\boldsymbol{c}$ 会不断增大. 当输入序列的长度非常大时，记忆单元的容量会饱和，从而大大降低 LSTM 模型的性能.

peephole 连接 另外一种变体是三个门不但依赖于输入 $\boldsymbol{x}_t$ 和上一时刻的隐状态 $\boldsymbol{h}_{t-1}$，也依赖于上一个时刻的记忆单元 $\boldsymbol{c}_{t-1}$，即

$$\boldsymbol{i}_t = \sigma(\boldsymbol{W}_i\boldsymbol{x}_t + \boldsymbol{U}_i\boldsymbol{h}_{t-1} + \boldsymbol{V}_i\boldsymbol{c}_{t-1} + \boldsymbol{b}_i), \tag{6.61}$$

$$\boldsymbol{f}_t = \sigma(\boldsymbol{W}_f\boldsymbol{x}_t + \boldsymbol{U}_f\boldsymbol{h}_{t-1} + \boldsymbol{V}_f\boldsymbol{c}_{t-1} + \boldsymbol{b}_f), \tag{6.62}$$

$$\boldsymbol{o}_t = \sigma(\boldsymbol{W}_o\boldsymbol{x}_t + \boldsymbol{U}_o\boldsymbol{h}_{t-1} + \boldsymbol{V}_o\boldsymbol{c}_t + \boldsymbol{b}_o), \tag{6.63}$$

其中 $\boldsymbol{V}_i, \boldsymbol{V}_f$ 和 $\boldsymbol{V}_o$ 为对角矩阵.

耦合输入门和遗忘门 LSTM 网络中的输入门和遗忘门有些互补关系，因此同时用两个门比较冗余. 为了减少 LSTM 网络的计算复杂度，将这两门合并为一个门. 令 $\boldsymbol{f}_t = \boldsymbol{1} - \boldsymbol{i}_t$，内部状态的更新方式为

$$\boldsymbol{c}_t = (\boldsymbol{1} - \boldsymbol{i}_t) \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t. \tag{6.64}$$

6.6.3 门控循环单元网络

门控循环单元（Gated Recurrent Unit，GRU）网络 [Cho et al., 2014; Chung et al., 2014] 是一种比 LSTM 网络更加简单的循环神经网络.

GRU 网络引入门控机制来控制信息更新的方式. 和 LSTM 不同，GRU 不引入额外的记忆单元，GRU 网络也是在公式 (6.50) 的基础上引入一个更新门（Update Gate）来控制当前状态需要从历史状态中保留多少信息（不经过非线性变换），以及需要从候选状态中接受多少新信息，即

$$\boldsymbol{h}_t = \boldsymbol{z}_t \odot \boldsymbol{h}_{t-1} + (\boldsymbol{1} - \boldsymbol{z}_t) \odot g(\boldsymbol{x}_t, \boldsymbol{h}_{t-1}; \theta), \tag{6.65}$$

其中 $\boldsymbol{z}_t \in [0, 1]^D$ 为更新门：

$$\boldsymbol{z}_t = \sigma\big(\boldsymbol{W}_z\boldsymbol{x}_t + \boldsymbol{U}_z\boldsymbol{h}_{t-1} + \boldsymbol{b}_z\big). \tag{6.66}$$

公式 (6.66)~公式 (6.68) 中的 $\boldsymbol{W}_*, \boldsymbol{U}_*, \boldsymbol{b}_*$ 为可学习的网络参数，其中 $* \in \{b, r, z\}$.

在 LSTM 网络中，输入门和遗忘门是互补关系，具有一定的冗余性. GRU 网络直接使用一个门来控制输入和遗忘之间的平衡. 当 $\boldsymbol{z}_t = \boldsymbol{0}$ 时，当前状态 $\boldsymbol{h}_t$ 和前一时刻的状态 $\boldsymbol{h}_{t-1}$ 之间为非线性函数关系；当 $\boldsymbol{z}_t = \boldsymbol{1}$ 时，$\boldsymbol{h}_t$ 和 $\boldsymbol{h}_{t-1}$ 之间为线性函数关系.

在 GRU 网络中，函数 $g(\boldsymbol{x}_t, \boldsymbol{h}_{t-1}; \theta)$ 的定义为

$$\tilde{\boldsymbol{h}}_t = \tanh\Big(\boldsymbol{W}_h\boldsymbol{x}_t + \boldsymbol{U}_h(\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1}) + \boldsymbol{b}_h\Big), \tag{6.67}$$

这里使用 Tanh 函数是由于其导数有比较大的值域，能够缓解梯度消失问题.

其中 $\tilde{\boldsymbol{h}}_t$ 表示当前时刻的候选状态，$\boldsymbol{r}_t \in [0,1]^D$ 为重置门（Reset Gate）

$$\boldsymbol{r}_t = \sigma(\boldsymbol{W}_r \boldsymbol{x}_t + \boldsymbol{U}_r \boldsymbol{h}_{t-1} + \boldsymbol{b}_r), \tag{6.68}$$

用来控制候选状态$\tilde{\boldsymbol{h}}_t$ 的计算是否依赖上一时刻的状态 $\boldsymbol{h}_{t-1}$.

当 $\boldsymbol{r}_t = 0$ 时，候选状态 $\tilde{\boldsymbol{h}}_t = \tanh(\boldsymbol{W}_c \boldsymbol{x}_t + \boldsymbol{b})$ 只和当前输入 $\boldsymbol{x}_t$ 相关，和历史状态无关. 当 $\boldsymbol{r}_t = 1$ 时，候选状态 $\tilde{\boldsymbol{h}}_t = \tanh(\boldsymbol{W}_h \boldsymbol{x}_t + \boldsymbol{U}_h \boldsymbol{h}_{t-1} + \boldsymbol{b}_h)$ 和当前输入 $\boldsymbol{x}_t$ 以及历史状态 $\boldsymbol{h}_{t-1}$ 相关，和简单循环网络一致.

综上，GRU 网络的状态更新方式为

$$\boldsymbol{h}_t = \boldsymbol{z}_t \odot \boldsymbol{h}_{t-1} + (1 - \boldsymbol{z}_t) \odot \tilde{\boldsymbol{h}}_t. \tag{6.69}$$

可以看出，当 $\boldsymbol{z}_t = \boldsymbol{0}, \boldsymbol{r} = \boldsymbol{1}$ 时，GRU 网络退化为简单循环网络；若 $\boldsymbol{z}_t = \boldsymbol{0}, \boldsymbol{r} = \boldsymbol{0}$ 时，当前状态 $\boldsymbol{h}_t$ 只和当前输入 $\boldsymbol{x}_t$ 相关，和历史状态 $\boldsymbol{h}_{t-1}$ 无关. 当 $\boldsymbol{z}_t = \boldsymbol{1}$ 时，当前状态 $\boldsymbol{h}_t = \boldsymbol{h}_{t-1}$ 等于上一时刻状态 $\boldsymbol{h}_{t-1}$，和当前输入 $\boldsymbol{x}_t$ 无关.

图6.8给出了 GRU 网络的循环单元结构.

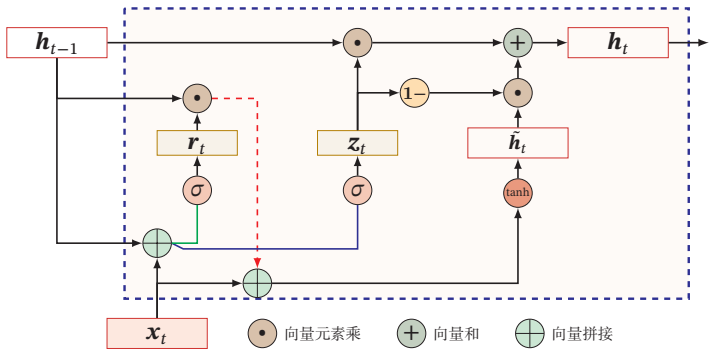


图 6.8 GRU 网络的循环单元结构

6.7 深层循环神经网络

如果将深度定义为网络中信息传递路径长度的话，循环神经网络可以看作既“深”又“浅”的网络. 一方面来说，如果我们把循环网络按时间展开，长时间间隔的状态之间的路径很长，循环网络可以看作一个非常深的网络. 从另一方面来说，如果同一时刻网络输入到输出之间的路径 $\boldsymbol{x}_t \to \boldsymbol{y}_t$，这个网络是非常浅的.

因此，我们可以增加循环神经网络的深度从而增强循环神经网络的能力. 增加循环神经网络的深度主要是增加同一时刻网络输入到输出之间的路径 $\boldsymbol{x}_t \to \boldsymbol{y}_t$，比如增加隐状态到输出 $\boldsymbol{h}_t \to \boldsymbol{y}_t$，以及输入到隐状态 $\boldsymbol{x}_t \to \boldsymbol{h}_t$ 之间的路径的深度.

6.7.1 堆叠循环神经网络

一种常见的增加循环神经网络深度的做法是将多个循环网络堆叠起来，称为堆叠循环神经网络（Stacked Recurrent Neural Network，SRNN）. 一个堆叠的简单循环网络（Stacked SRN）也称为循环多层感知器（Recurrent Multi-Layer Perceptron，RMLP）[Parlos et al., 1991].

参见习题6-6.

图6.9给出了按时间展开的堆叠循环神经网络. 第 l 层网络的输入是第 $l-1$ 层网络的输出. 我们定义 $\boldsymbol{h}_t^{(l)}$ 为在时刻 t 时第 l 层的隐状态

$$\boldsymbol{h}_t^{(l)} = f(\boldsymbol{U}^{(l)}\boldsymbol{h}_{t-1}^{(l)} + \boldsymbol{W}^{(l)}\boldsymbol{h}_t^{(l-1)} + \boldsymbol{b}^{(l)}), \tag{6.70}$$

其中 $\boldsymbol{U}^{(l)}$、$\boldsymbol{W}^{(l)}$ 和 $\boldsymbol{b}^{(l)}$ 为权重矩阵和偏置向量，$\boldsymbol{h}_t^{(0)} = \boldsymbol{x}_t$.

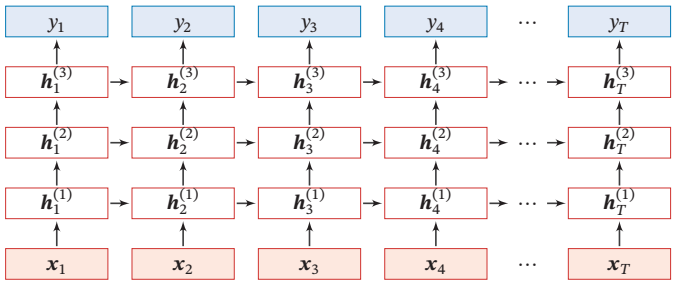


图 6.9 按时间展开的堆叠循环神经网络

6.7.2 双向循环神经网络

在有些任务中，一个时刻的输出不但和过去时刻的信息有关，也和后续时刻的信息有关. 比如给定一个句子，其中一个词的词性由它的上下文决定，即包含左右两边的信息. 因此，在这些任务中，我们可以增加一个按照时间的逆序来传递信息的网络层，来增强网络的能力.

双向循环神经网络（Bidirectional Recurrent Neural Network，Bi-RNN）由两层循环神经网络组成，它们的输入相同，只是信息传递的方向不同.

假设第 1 层按时间顺序，第 2 层按时间逆序，在时刻 t 时的隐状态定义为 $\boldsymbol{h}_t^{(1)}$ 和 $\boldsymbol{h}_t^{(2)}$，则

$$\boldsymbol{h}_t^{(1)} = f(\boldsymbol{U}^{(1)}\boldsymbol{h}_{t-1}^{(1)} + \boldsymbol{W}^{(1)}\boldsymbol{x}_t + \boldsymbol{b}^{(1)}), \tag{6.71}$$

$$\boldsymbol{h}_t^{(2)} = f(\boldsymbol{U}^{(2)}\boldsymbol{h}_{t+1}^{(2)} + \boldsymbol{W}^{(2)}\boldsymbol{x}_t + \boldsymbol{b}^{(2)}), \tag{6.72}$$

$$\boldsymbol{h}_t = \boldsymbol{h}_t^{(1)} \oplus \boldsymbol{h}_t^{(2)}, \tag{6.73}$$

其中 $\oplus$ 为向量拼接操作.

图6.10给出了按时间展开的双向循环神经网络.

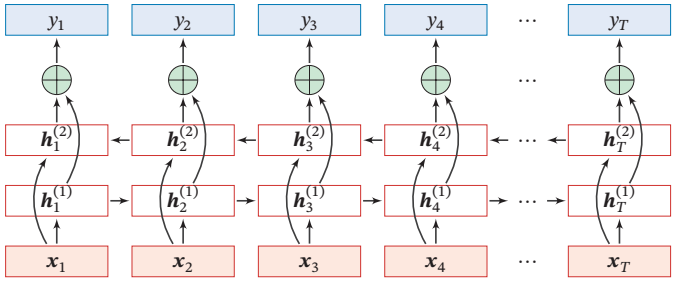


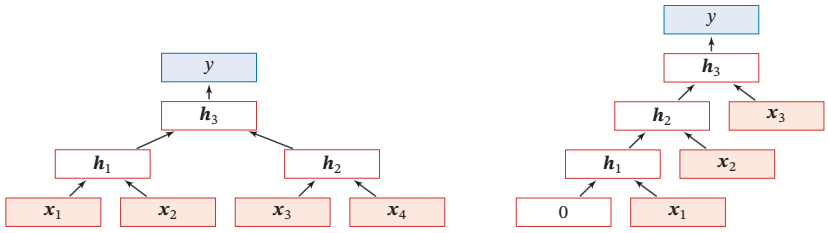
图 6.10 按时间展开的双向循环神经网络

6.8 扩展到图结构

如果将循环神经网络按时间展开，每个时刻的隐状态 $\boldsymbol{h}_t$ 看作一个节点，那么这些节点构成一个链式结构，每个节点 t 都收到其父节点的消息（Message），更新自己的状态，并传递给其子节点. 而链式结构是一种特殊的图结构，我们可以比较容易地将这种消息传递（Message Passing）的思想扩展到任意的图结构上.

6.8.1 递归神经网络

递归神经网络（Recursive Neural Network，RecNN）是循环神经网络在有向无循环图上的扩展 [Pollack, 1990]. 递归神经网络的一般结构为树状的层次结构，如图6.11a所示.



(a) 一般结构 (b) 退化结构

图 6.11 递归神经网络

以图6.11a中的结构为例，有三个隐藏层 $\boldsymbol{h}_1$、$\boldsymbol{h}_2$ 和 $\boldsymbol{h}_3$，其中 $\boldsymbol{h}_1$ 由两个输入层 $\boldsymbol{x}_1$ 和 $\boldsymbol{x}_2$ 计算得到，$\boldsymbol{h}_2$ 由另外两个输入层 $\boldsymbol{x}_3$ 和 $\boldsymbol{x}_4$ 计算得到，$\boldsymbol{h}_3$ 由两个隐藏层 $\boldsymbol{h}_1$ 和 $\boldsymbol{h}_2$ 计算得到.

对于一个节点 $\boldsymbol{h}_i$，它可以接受来自父节点集合 π_i 中所有节点的消息，并更

新自己的状态.

$$\boldsymbol{h}_i = f(\boldsymbol{h}_{\pi_i}), \tag{6.74}$$

其中 $\boldsymbol{h}_{\pi_i}$ 表示集合 π_i 中所有节点状态的拼接，$f(\cdot)$ 是一个和节点位置无关的非线性函数，可以为一个单层的前馈神经网络. 比如图6.11a所示的递归神经网络具体可以写为

$$\boldsymbol{h}_1 = \sigma(\boldsymbol{W}\begin{bmatrix}\boldsymbol{x}_1\\ \boldsymbol{x}_2\end{bmatrix} + \boldsymbol{b}), \tag{6.75}$$

$$\boldsymbol{h}_2 = \sigma(\boldsymbol{W}\begin{bmatrix}\boldsymbol{x}_3\\ \boldsymbol{x}_4\end{bmatrix} + \boldsymbol{b}), \tag{6.76}$$

$$\boldsymbol{h}_3 = \sigma(\boldsymbol{W}\begin{bmatrix}\boldsymbol{h}_1\\ \boldsymbol{h}_2\end{bmatrix} + \boldsymbol{b}), \tag{6.77}$$

其中 $\sigma(\cdot)$ 表示非线性激活函数，$\boldsymbol{W}$ 和 $\boldsymbol{b}$ 是可学习的参数. 同样，输出层 y 可以为一个分类器，比如

$$y = g(\boldsymbol{W}'\boldsymbol{h}_3 + \boldsymbol{b}'), \tag{6.78}$$

其中 $g(\cdot)$ 为分类器，$\boldsymbol{W}'$ 和 $\boldsymbol{b}'$ 为分类器的参数.

当递归神经网络的结构退化为线性序列结构（见图6.11b）时，递归神经网络就等价于简单循环网络.

参见习题6-7.

递归神经网络主要用来建模自然语言句子的语义[Socher et al., 2011, 2013]. 给定一个句子的语法结构（一般为树状结构），可以使用递归神经网络来按照句法的组合关系来合成一个句子的语义. 句子中每个短语成分又可以分成一些子成分，即每个短语的语义都可以由它的子成分语义组合而来，并进而合成整句的语义.

同样，我们也可以用门控机制来改进递归神经网络中的长距离依赖问题，比如树结构的长短期记忆模型（Tree-Structured LSTM）[Tai et al., 2015; Zhu et al., 2015] 就是将 LSTM 模型的思想应用到树结构的网络中，来实现更灵活的组合函数.

6.8.2 图神经网络

在实际应用中，很多数据是图结构的，比如知识图谱、社交网络、分子网络等. 而前馈网络和反馈网络很难处理图结构的数据.

图神经网络（Graph Neural Network，GNN）是将消息传递的思想扩展到图结构数据上的神经网络.

对于一个任意的图结构 $G(\mathcal{V},\mathcal{E})$，其中 $\mathcal{V}$ 表示节点集合，$\mathcal{E}$ 表示边集合. 每条边表示两个节点之间的依赖关系. 节点之间的连接可以是有向的，也可以是无向的. 图中每个节点 v 都用一组神经元来表示其状态 $\boldsymbol{h}^{(v)}$，初始状态可以为节点 v 的输入特征 $\boldsymbol{x}^{(v)}$. 每个节点可以收到来自相邻节点的消息，并更新自己的状态.

$$\boldsymbol{m}_t^{(v)} = \sum_{u\in\mathcal{N}(v)} f\big(\boldsymbol{h}_{t-1}^{(v)}, \boldsymbol{h}_{t-1}^{(u)}, \boldsymbol{e}^{(u,v)}\big), \tag{6.79}$$

$$\boldsymbol{h}_t^{(v)} = g\big(\boldsymbol{h}_{t-1}^{(v)}, \boldsymbol{m}_t^{(v)}\big), \tag{6.80}$$

其中 $\mathcal{N}(v)$ 表示节点 v 的邻居，$\boldsymbol{m}_t^{(v)}$ 表示在第 t 时刻节点 v 收到的信息，$\boldsymbol{e}^{(u,v)}$ 为边 $e^{(u,v)}$ 上的特征.

公式(6.79)和公式(6.80)是一种同步的更新方式，所有的结构同时接受信息并更新自己的状态. 而对于有向图来说，使用异步的更新方式会更有效率，比如循环神经网络或递归神经网络. 在整个图更新 T 次后，可以通过一个读出函数（Readout Function）$g(\cdot)$ 来得到整个网络的表示：

$$\boldsymbol{o}_t = g\Big(\{\boldsymbol{h}_T^{(v)} | v \in \mathcal{V}\}\Big). \tag{6.81}$$

6.9 总结和深入阅读

循环神经网络可以建模时间序列数据之间的相关性. 和延时神经网络[Lang et al., 1990; Waibel et al., 1989]以及有外部输入的非线性自回归模型[Leontaritis et al., 1985]相比，循环神经网络可以更方便地建模长时间间隔的相关性.

常用的循环神经网络的参数学习算法是 BPTT 算法[Werbos, 1990]，其计算时间和空间要求会随时间线性增长. 为了提高效率，当输入序列的长度比较大时，可以使用带截断（truncated）的 BPTT 算法[Williams et al., 1990]，只计算固定时间间隔内的梯度回传.

一个完全连接的循环神经网络有着强大的计算和表示能力，可以近似任何非线性动力系统以及图灵机，解决所有的可计算问题. 然而由于梯度爆炸和梯度消失问题，简单循环网络存在长期依赖问题[Bengio et al., 1994; Hochreiter et al., 2001]. 为了解决这个问题，人们对循环神经网络进行了很多的改进，其中最有效的改进方式为引入门控机制，比如LSTM 网络 [Gers et al., 2000; Hochreiter et al., 1997]和GRU 网络[Chung et al., 2014]. 当然还有一些其他方法，比如时钟循环神经网络（Clockwork RNN）[Koutnik et al., 2014]、乘法RNN[Sutskever et al., 2011; Wu et al., 2016]以及引入注意力机制等.

注意力机制参见第8.2节.

LSTM 网络是目前为止最成功的循环神经网络模型，成功应用在很多领域，比如语音识别、机器翻译 [Sutskever et al., 2014]、语音模型以及文本生成. LSTM 网络通过引入线性连接来缓解长距离依赖问题. 虽然 LSTM 网络取得了很大的成功，其结构的合理性一直受到广泛关注. 人们不断尝试对其进行改进来寻找最优结构，比如减少门的数量、提高并行能力等. 关于 LSTM 网络的分析可以参考文献 [Greff et al., 2017; Jozefowicz et al., 2015; Karpathy et al., 2015].

LSTM 网络的线性连接以及门控机制是一种十分有效的避免梯度消失问题的方法. 这种机制也可以用在深层的前馈网络中，比如残差网络 [He et al., 2016] 和高速网络 [Srivastava et al., 2015] 都通过引入线性连接来训练非常深的卷积网络. 对于循环神经网格，这种机制也可以用在非时间维度上，比如 Gird LSTM 网络 [Kalchbrenner et al., 2015]、Depth Gated RNN[Chung et al., 2015] 等.

此外，循环神经网络可以很容易地扩展到更广义的图结构数据上，称为图网络[Scarselli et al., 2009]. 递归神经网络是一种在有向无环图上的简单的图网络. 图网络是目前新兴的研究方向，还没有比较成熟的网络模型. 在不同的网络结构以及任务上，都有很多不同的具体实现方式. 其中比较有名的图网络模型包括 图卷积网络（Graph Convolutional Network，GCN）[Kipf et al., 2016]、图注意力网络（Graph Attention Network，GAT）[Veličković et al., 2017]、消息传递神经网络（Message Passing Neural Network，MPNN）[Gilmer et al., 2017] 等. 关于图网络的综述可以参考文献 [Battaglia et al., 2018].

习题

习题 6-1 分析延时神经网络、卷积神经网络和循环神经网络的异同点.

习题 6-2 推导公式 (6.40) 和公式 (6.41) 中的梯度.

习题 6-3 当使用公式 (6.50) 作为循环神经网络的状态更新公式时，分析其可能存在梯度爆炸的原因并给出解决方法.

习题 6-4 推导 LSTM 网络中参数的梯度，并分析其避免梯度消失的效果.

习题 6-5 推导 GRU 网络中参数的梯度，并分析其避免梯度消失的效果.

习题 6-6 除了堆叠循环神经网络外，还有什么结构可以增加循环神经网络深度？

习题 6-7 证明当递归神经网络的结构退化为线性序列结构时，递归神经网络就等价于简单循环神经网络.

参考文献

Battaglia P W, Hamrick J B, Bapst V, et al., 2018. Relational inductive biases, deep learning, and graph networks[J]. arXiv preprint arXiv:1806.01261.

Bengio Y, Simard P, Frasconi P, 1994. Learning long-term dependencies with gradient descent is difficult[J]. Neural Networks, IEEE Transactions on, 5(2):157-166.

Cho K, Van Merriënboer B, Gulcehre C, et al., 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation[J]. arXiv preprint arXiv:1406.1078.

Chung J, Gulcehre C, Cho K, et al., 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling[J]. arXiv preprint arXiv:1412.3555.

Chung J, Gulcehre C, Cho K, et al., 2015. Gated feedback recurrent neural networks[C]// International Conference on Machine Learning. 2067-2075.

Elman J L, 1990. Finding structure in time[J]. Cognitive science, 14(2):179-211.

Gers F A, Schmidhuber J, Cummins F, 2000. Learning to forget: Continual prediction with lstm[J]. Neural Computation.

Gilmer J, Schoenholz S S, Riley P F, et al., 2017. Neural message passing for quantum chemistry[J]. arXiv preprint arXiv:1704.01212.

Greff K, Srivastava R K, Koutník J, et al., 2017. Lstm: A search space odyssey[J]. IEEE transactions on neural networks and learning systems.

Haykin S, 2009. Neural networks and learning machines[M]. 3rd edition. Pearson.

He K, Zhang X, Ren S, et al., 2016. Deep residual learning for image recognition[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 770-778.

Hochreiter S, Schmidhuber J, 1997. Long short-term memory[J]. Neural computation, 9(8):1735-1780.

Hochreiter S, Bengio Y, Frasconi P, et al., 2001. Gradient flow in recurrent nets: The difficulty of learning longterm dependencies[M/OL]//Kolen J F, Kremer S C. A Field Guide to Dynamical Recurrent Networks. IEEE: 237-243. https://ieeexplore.ieee.org/document/5264952.

Jozefowicz R, Zaremba W, Sutskever I, 2015. An empirical exploration of recurrent network architectures[C]//Proceedings of the 32nd International Conference on Machine Learning. 2342-2350.

Kalchbrenner N, Danihelka I, Graves A, 2015. Grid long short-term memory[J]. arXiv preprint arXiv:1507.01526.

Karpathy A, Johnson J, Fei-Fei L, 2015. Visualizing and understanding recurrent networks[J]. arXiv preprint arXiv:1506.02078.

Kipf T N, Welling M, 2016. Semi-supervised classification with graph convolutional networks[J]. arXiv preprint arXiv:1609.02907.

Koutnik J, Greff K, Gomez F, et al., 2014. A clockwork rnn[C]//Proceedings of The 31st International Conference on Machine Learning. 1863-1871.

Lang K J, Waibel A H, Hinton G E, 1990. A time-delay neural network architecture for isolated word recognition[J]. Neural networks, 3(1):23-43.

Leontaritis I, Billings S A, 1985. Input-output parametric models for non-linear systems part i: deterministic non-linear systems[J]. International journal of control, 41(2):303-328.

Parlos A, Atiya A, Chong K, et al., 1991. Recurrent multilayer perceptron for nonlinear system identification[C]//International Joint Conference on Neural Networks: volume 2. IEEE: 537-540.

Pollack J B, 1990. Recursive distributed representations[J]. Artificial Intelligence, 46(1):77-105.

Scarselli F, Gori M, Tsoi A C, et al., 2009. The graph neural network model[J]. IEEE Transactions on Neural Networks, 20(1):61-80.

Schäfer A M, Zimmermann H G, 2006. Recurrent neural networks are universal approximators[C]// International Conference on Artificial Neural Networks. Springer: 632-640.

Siegelmann H T, Sontag E D, 1991. Turing computability with neural nets[J]. Applied Mathematics Letters, 4(6):77-80.

Socher R, Lin C C, Manning C, et al., 2011. Parsing natural scenes and natural language with recursive neural networks[C]//Proceedings of the International Conference on Machine Learning.

Socher R, Perelygin A, Wu J Y, et al., 2013. Recursive deep models for semantic compositionality over a sentiment treebank[C]//Proceedings of EMNLP.

Srivastava R K, Greff K, Schmidhuber J, 2015. Highway networks[J]. arXiv preprint arXiv:1505.00387.

Sutskever I, Martens J, Hinton G E, 2011. Generating text with recurrent neural networks[C]// Proceedings of the 28th International Conference on Machine Learning. 1017-1024.

Sutskever I, Vinyals O, Le Q V, 2014. Sequence to sequence learning with neural networks[C]// Advances in Neural Information Processing Systems. 3104-3112.

Tai K S, Socher R, Manning C D, 2015. Improved semantic representations from tree-structured long short-term memory networks[C]//Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics.

Veličković P, Cucurull G, Casanova A, et al., 2017. Graph attention networks[J]. arXiv preprint arXiv:1710.10903.

Waibel A, Hanazawa T, Hinton G, et al., 1989. Phoneme recognition using time-delay neural networks[J]. IEEE transactions on acoustics, speech, and signal processing, 37(3):328-339.

Werbos P J, 1990. Backpropagation through time: what it does and how to do it[J]. Proceedings of the IEEE, 78(10):1550-1560.

Williams R J, Peng J, 1990. An efficient gradient-based algorithm for on-line training of recurrent network trajectories[J]. Neural computation, 2(4):490-501.

Williams R J, Zipser D, 1995. Gradient-based learning algorithms for recurrent networks and their computational complexity[J]. Backpropagation: Theory, architectures, and applications, 1:433-486.

Wu Y, Zhang S, Zhang Y, et al., 2016. On multiplicative integration with recurrent neural networks [C]//Advances in neural information processing systems. 2856-2864.

Zhu X, Sobihani P, Guo H, 2015. Long short-term memory over recursive structures[C]//Proceedings of LCML. 1604-1612.